

National Institute of Technology Karnataka, Surathkal
Department of Computer Science and Engineering

CS255 — Data Communications & Networks

Mini-Project Progress Report – I

Packet-Level Forward Error Correction
Using GF(256) Cauchy Codes for
5G Ultra-Reliable Low-Latency
Communication (URLLC)
at the Application Layer

Project Repository

[somyaknotfound/fec_project](https://github.com/somyaknotfound/fec_project)

Students: Somyak Priyadarshi Mohanta (241CS257)
Shivam Kumar (241CS255)

Department: Computer Science and Engineering

Institute: National Institute of Technology Karnataka, Surathkal

Guide: Prof. B. R. Chandavarkar

Department: Computer Science and Engineering, NITK

Abstract

Fifth-generation (5G) Ultra-Reliable Low-Latency Communication (URLLC) mandates end-to-end latency below 1 ms and packet delivery reliability exceeding 99.999%. Hybrid Automatic Repeat Request (HARQ), the conventional remedy for lost packets, requires at least one additional round-trip delay and is therefore fundamentally incompatible with URLLC latency requirements. Physical-layer LDPC and Polar codes correct bit errors but cannot recover entire UDP packets lost to congestion or handover gaps. Existing application-layer alternatives—Raptor-Q and RLNC—introduce variable-latency or computationally unbounded decoding, unsuitable for hard real-time systems.

This paper presents a packet-level Forward Error Correction system at the application layer using $GF(2^8)$ Cauchy matrix encoding and Gaussian elimination decoding over raw UDP sockets. A systematic ($n=8$, $k=4$) erasure code encodes every four data packets into eight (four data + four parity); any four surviving packets suffice to recover all original data—without retransmission. We evaluate the system through 4,500 FEC blocks spanning nine channel-loss rates (0%–50%) across three channel models: Bernoulli independent loss, Gilbert-Elliott two-state Markov burst loss, and MATLAB 5G Toolbox `nrTDLChannel` (TDL-A pedestrian and TDL-B vehicular scenarios). Results demonstrate complete elimination of application-level packet loss at channel loss rates up to 15%, over 86% PLR reduction at 30% channel loss, only ≈ 0.5 ms encoding overhead—well within the URLLC budget—and confirmation that the Gilbert-Elliott statistical model accurately approximates the MATLAB physics-based TDL-B channel (mean burst length ≈ 3.2 vs. 3.3 packets respectively). The implementation is validated at the protocol level using Linux `tc netem` and live Wireshark capture. Future work targets integration with an Open5GS 5G core and srsRAN for end-to-end over-the-air validation.

Keywords: 5G URLLC, Forward Error Correction, $GF(256)$, Cauchy Matrix, Erasure Coding, Gaussian Elimination, Gilbert-Elliott Channel, `nrTDLChannel`, MATLAB 5G Toolbox, UDP, Application-Layer FEC, Open5GS, srsRAN.

Contents

1	Introduction	3
1.1	Motivation: Application-Layer FEC	3
1.2	Problem Statement	4
1.3	Objectives	4
1.4	Organisation of the Report	4
2	Literature Review	4
2.1	HARQ and Physical-Layer FEC in 5G NR	4
2.2	Adaptive HARQ Schemes at the Link Layer	5
2.2.1	Adaptive HARQ with Reed-Solomon Codes	5
2.2.2	Three-Stage Adaptive HARQ with Two RS Codes	5
2.2.3	Adaptive HARQ with BCH Codes	6
2.2.4	Dynamic HARQ (D-HARQ) with Turbo Codes and Selective Soft Combining	6
2.3	Application-Layer Erasure Coding for IP/UDP Networks	6
2.3.1	Cauchy Matrix Erasure Codes	6
2.3.2	Raptor-Q Codes	7
2.3.3	QUIC-FEC	7
2.3.4	Spatially-Coupled LDPC for URLLC	7
2.4	Comparison of Existing Approaches	8
3	Proposed Methodology	8
3.1	System Overview	8
3.2	GF(256) Arithmetic Foundation	9
3.3	Systematic Cauchy Encoder	9
3.4	Gaussian Elimination Decoder	10
3.5	FEC Packet Header Format	10
3.6	Channel Models	11
3.7	MATLAB–Python Integration Architecture	11
3.8	5G Latency Model	12
3.9	Software Architecture	12
3.10	Key Engineering Challenges and Solutions	14
3.10.1	Identical Parity Packets (Critical Bug)	14
3.10.2	Wireshark-Invisible Loss	14
3.10.3	Decoder Matrix Singularity	14
3.10.4	MATLAB CSV Path and Filename Compatibility	14
3.11	Planned Extension: Open5GS + srsRAN Integration	15
4	Results and Analysis	15
4.1	Experimental Setup	15
4.2	Performance Metrics	16

4.3	Unit Test Validation	16
4.4	Packet Loss Rate — All Channel Models	17
4.5	Block Recovery Rate	18
4.6	Burst Loss Distribution — Gilbert-Elliott vs. MATLAB TDL-B	19
4.7	Effective Throughput	20
4.8	Latency Analysis	21
4.9	Summary Comparison Across All Channel Models	22
4.10	Wireshark Protocol-Level Validation	23
5	Conclusions and Future Work	26
5.1	Summary of Achievements	26
5.2	Limitations	27
5.3	Future Work	27

1. Introduction

The fifth generation of mobile networks introduces three distinct service categories: enhanced Mobile Broadband (eMBB), massive Machine-Type Communication (mMTC), and Ultra-Reliable Low-Latency Communication (URLLC) [1]. URLLC is the most demanding category, mandating simultaneous guarantees of end-to-end latency ≤ 1 ms and packet delivery reliability $\geq 99.999\%$ —one lost packet per hundred thousand—for safety-critical applications such as industrial automation, remote surgery, and autonomous vehicles. Table 1 summarises the 3GPP-specified requirements for representative use cases.

Table 1: 5G URLLC Latency and Reliability Requirements (3GPP TR 22.862 [1])

Use Case	Max. Latency	Min. Reliability	Example
Industrial Automation	<1 ms	99.999%	Factory robotics
Remote Surgery	<1 ms	99.9999%	Tele-operation
Autonomous Vehicles	<5 ms	99.999%	V2X communication
IoT Sensor Networks	<10 ms	99.9%	Smart sensors

At the physical layer, 5G New Radio (NR) employs LDPC codes for eMBB data channels and Polar codes for control channels as specified in 3GPP TS 38.212 [2]. These codes correct *bit errors* within a single Transmission Time Interval (TTI) as short as 0.125 ms. However, they provide no mechanism for recovering *entire UDP packets* lost due to network congestion, scheduling gaps during handovers, or application buffer overflow—phenomena that occur above the physical layer.

The conventional remedy is Hybrid Automatic Repeat Request (HARQ), which combines ARQ retransmission with forward error correction [4]. Even in 5G NR with 15 kHz sub-carrier spacing, the minimum HARQ round-trip occupies approximately 4 TTIs (≈ 4 ms)—four times the entire URLLC latency budget [13]. This makes HARQ fundamentally incompatible with URLLC for any scenario requiring even a single retransmission.

1.1. Motivation: Application-Layer FEC

Forward Error Correction at the application layer [7, 3] pre-emptively transmits redundant parity packets alongside original data. The receiver algebraically reconstructs lost packets from surviving ones—without feedback, without retransmission, and without additional round-trip delay. This erasure-coding paradigm is the theoretical basis for QR codes, RAID-6 storage, and satellite communication FEC [5, 22].

The mathematical foundation is $\text{GF}(2^8)$ —the finite field with 256 elements—whose elements map naturally to bytes, making byte-level packet erasure coding both algebraically

correct and computationally efficient [5, 14]. Cauchy matrices over $\text{GF}(2^8)$ are proven Maximum Distance Separable (MDS): every square sub-matrix is invertible, guaranteeing recovery from *any* r losses per block regardless of which packets are lost [6, 7]. Gaussian elimination provides an exact, bounded-time decoder.

1.2. Problem Statement

Given the strict latency constraint of 5G URLLC (≤ 1 ms end-to-end), design and implement an application-layer FEC system over raw UDP sockets that recovers from any combination of up to r lost packets per block without retransmission, with bounded encoding and decoding latency well below the URLLC budget, and validate the system on a live 5G testbed.

1.3. Objectives

- O1.** Implement a complete $\text{GF}(256)$ FEC stack from mathematical foundations through UDP transport, including systematic Cauchy encoding and Gauss-Jordan decoding.
- O2.** Characterise FEC performance across nine channel loss rates (0–50%) under three channel models: Bernoulli independent loss, Gilbert-Elliott two-state Markov burst loss, and MATLAB 5G Toolbox `nrTDLChannel` (TDL-A and TDL-B scenarios), through 4,500 controlled FEC blocks.
- O3.** Extend the validated application-layer FEC to a 5G testbed using Open5GS core and srsRAN software-defined radio for end-to-end over-the-air recovery validation.

1.4. Organisation of the Report

Section 2 reviews related work. Section 3 describes the proposed methodology. Section 4 presents experimental results. Section 5 concludes with future work.

2. Literature Review

This section reviews three categories of related work: (A) HARQ and physical-layer FEC in 5G; (B) adaptive HARQ schemes at the link layer; and (C) application-layer erasure coding for IP/UDP networks. Each subsection is followed by a pros and cons analysis. A consolidated comparison is given in Table 2.

2.1. HARQ and Physical-Layer FEC in 5G NR

Standard ARQ retransmits an entire packet on error detection, incurring at least one additional round-trip delay. In 5G NR the minimum HARQ round-trip at 15 kHz sub-carrier spacing is ≈ 4 ms [2, 4]. Two HARQ variants exist: *Chase Combining* (CC-HARQ) retransmits identical copies and combines them to improve signal energy; *Incremental*

Redundancy (IR-HARQ) transmits additional parity bits on each round, gradually improving the effective code rate [4]. Both still require at least one additional round trip on first-transmission failure.

3GPP TS 38.212 specifies LDPC codes for data channels and Polar codes for control channels in 5G NR [2]. Both operate at the bit level within a single TTI and correct bit errors due to AWGN and fading. They do not recover entire lost packets at the IP or transport layer.

Pros: Robust bit-error correction; standardised and hardware-accelerated in 5G chipsets.

Cons: Minimum retransmission latency (≈ 4 ms) exceeds the URLLC budget; cannot recover from IP-layer packet loss.

2.2. Adaptive HARQ Schemes at the Link Layer

2.2.1. Adaptive HARQ with Reed-Solomon Codes

Fidler et al. [16] proposed switching between pure ARQ and HARQ with RS codes based on counting consecutive ACKs and NACKs. A threshold T sets the number of consecutive ACKs required to transition from HARQ back to ARQ; a single NACK triggers immediate return to HARQ. The scheme uses one RS code rate throughout. Figure 1 illustrates the switching logic.

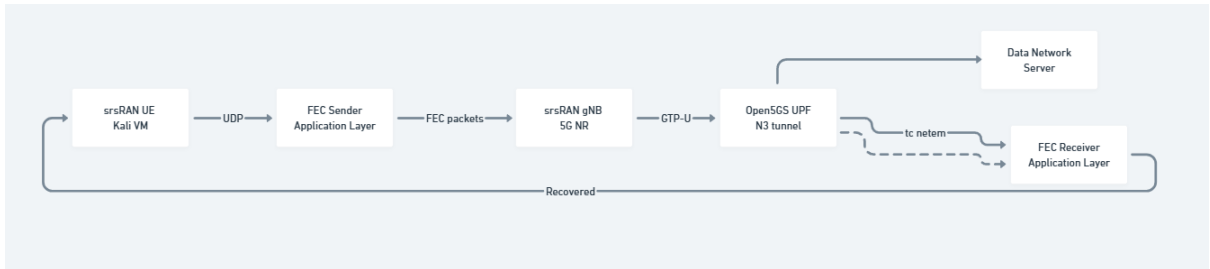


Figure 1: Adaptive HARQ with RS codes: ACK/NACK-based two-state switching [16].

Pros: Asymmetric switching responds quickly to channel degradation.

Cons: Only two states; limited adaptability at moderate error rates; no very-low-SNR handling.

2.2.2. Three-Stage Adaptive HARQ with Two RS Codes

Martinovic et al. [17] extended the two-state scheme to three stages: pure ARQ-GBN, HARQ1 with RS (511, 383, 64), and HARQ2 with RS (511, 255, 128). The transmitter controls upward transitions (ARQ→HARQ1→HARQ2) based on NACK counts; the receiver controls downward transitions using ACK+ signals (no errors in the last W packets) [17].

Pros: Higher throughput at moderate error rates than two-state schemes; sliding window.

Cons: Does not handle very low SNR; increased latency for real-time traffic; limited

capability against random errors.

2.2.3. Adaptive HARQ with BCH Codes

Kotuliakova et al. [18] presented a three-state scheme (ARQ-GBN, HARQ1, HARQ2) that selects between two BCH codes based on estimated SNR thresholds. Transitions are driven by SNR estimation rather than ACK/NACK counts. The GBN protocol is used in all modes.

Pros: Smooth transitions across a wide range of channel conditions; reduced latency vs. RS schemes.

Cons: Near-zero throughput at SNR < 3.47 dB; still requires retransmission.

2.2.4. Dynamic HARQ (D-HARQ) with Turbo Codes and Selective Soft Combining

Chintha et al. [15] proposed D-HARQ, which adds a fourth state using HARQ with turbo codes (code rate 1/3) specifically for very low SNR conditions (0–3.47 dB) where BCH-based HARQ yields near-zero throughput. An Error Vector Magnitude (EVM) algorithm estimates SNR at the receiver; soft combining is applied selectively only when SNR < 17.46 dB, reducing complexity at high SNR. Simulations in a Land-Mobile-Satellite channel show throughput improvement from 0 to 0.467 bpcu at low SNR.

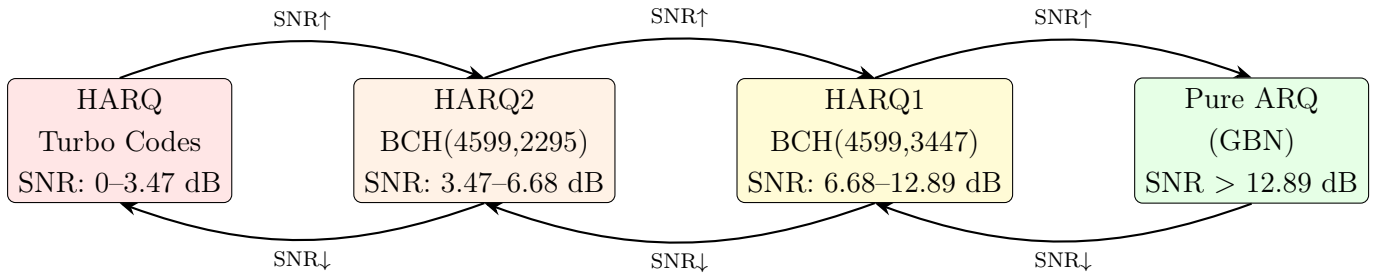


Figure 2: D-HARQ four-state switching across SNR ranges [15].

Pros: Resolves the zero-throughput gap at very low SNR; selective soft combining reduces complexity overhead at high SNR.

Cons: Operates at the link layer for bit-error correction; does not recover entire UDP packets lost at the IP layer; turbo decoding complexity is high at low SNR.

2.3. Application-Layer Erasure Coding for IP/UDP Networks

2.3.1. Cauchy Matrix Erasure Codes

Rizzo [7] proposed Cauchy matrices over $\text{GF}(2^8)$ as the basis for practical application-layer erasure codes in IP networks. A Cauchy matrix C of dimensions $m \times n$ is defined as:

$$C_{j,i} = \frac{1}{x_j \oplus y_i} \quad \text{in } \text{GF}(q) \quad (1)$$

where $\{x_j\}$ and $\{y_i\}$ are disjoint sets such that $x_j \oplus y_i \neq 0$ for all j, i . Every square sub-matrix of a Cauchy matrix is invertible over a finite field of sufficient size [6], guaranteeing MDS recovery.

2.3.2. Raptor-Q Codes

Raptor-Q codes use a fountain code architecture with LT codes and pre-coding stages [23]. Ramly et al. [10] demonstrated Raptor-Q at the application layer achieving the 1 ms end-to-end delay target for URLLC with mid-band 5G spectrum. However, the iterative belief-propagation decoder has variable latency that is difficult to bound in hard real-time systems.

Pros: Near-capacity performance; rateless (can generate unlimited parity symbols).

Cons: Variable and potentially unbounded decoding latency; high implementation complexity.

2.3.3. QUIC-FEC

Michel et al. [12] integrated forward erasure correction into the QUIC transport protocol, showing that application-layer redundancy complements physical-layer coding. QUIC-FEC requires transport-layer modification and cannot be deployed over raw UDP.

Pros: Complements physical-layer FEC; works within existing QUIC congestion control.

Cons: Requires QUIC transport; not applicable to bare UDP URLLC traffic.

2.3.4. Spatially-Coupled LDPC for URLLC

Konfe et al. [11] proposed spatially-coupled LDPC codes extending physical-layer FEC to gain up to 0.8 dB additional coding gain for URLLC scenarios. This scheme operates below the transport layer and does not address packet-level erasure recovery over UDP.

2.4. Comparison of Existing Approaches

Table 2: Comparison of Existing Error Control Approaches for 5G URLLC

Approach	Layer	UDP Packet Recovery	Retransmission	Bounded Latency
LDPC/Polar (5G NR) [2]	Physical	No	No	Yes
CC-HARQ / IR-HARQ [4]	Link	No	Yes (≥ 4 ms RTT)	No
Adaptive HARQ RS [16]	Link	No	Yes	No
Two-RS HARQ [17]	Link	No	Yes	No
BCH Adaptive HARQ [18]	Link	No	Yes	No
D-HARQ Turbo+BCH [15]	Link	No	Yes	No
Raptor-Q App. FEC [10]	Application	Yes	No	No (Variable)
QUIC-FEC [12]	Transport	Partial	No	Yes
SC-LDPC [11]	Physical	No	No	Yes
Proposed: GF(256) Cauchy FEC	Application	Yes	No	Yes (≈ 0.5 ms)

The comparison reveals a clear gap: no existing approach simultaneously provides UDP packet-level erasure recovery, zero retransmission, and bounded sub-millisecond decoding latency over raw UDP sockets. The proposed GF(256) Cauchy FEC system is designed to fill this gap.

3. Proposed Methodology

3.1. System Overview

The proposed system encodes blocks of $k=4$ data packets into $n=8$ packets ($k=4$ data + $r=4$ parity), transmits all eight over UDP, and reconstructs any lost combination of up to four packets at the receiver—without retransmission. The code rate is $R_c = k/n = 0.5$

and the maximum tolerable loss per block is $r/n = 50\%$. Figure 3 shows the end-to-end data flow.

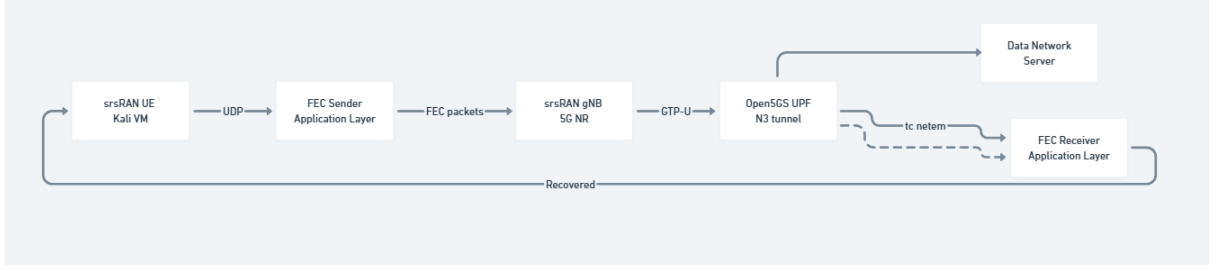


Figure 3: System architecture: application-layer FEC over UDP with 5G channel simulation. Three channel models are evaluated: Bernoulli (Python), Gilbert-Elliott (Python), and MATLAB 5G Toolbox nrTDLChannel.

3.2. GF(256) Arithmetic Foundation

All encoding and decoding arithmetic operates in $\text{GF}(2^8)$, where elements correspond to bytes (0–255). Addition is bitwise XOR (no carry). Multiplication uses precomputed log/exp tables for $O(1)$ operations:

$$a \cdot b = \exp[\log a + \log b \bmod 255], \quad a, b \neq 0 \quad (2)$$

$$a^{-1} = \exp[255 - \log a] \quad (3)$$

The irreducible polynomial is $x^8 + x^4 + x^3 + x^2 + 1$ (0x11D), identical to AES [14]. The double-length exp table (512 entries) avoids modular indexing during multiplication, giving a practical speedup of $\approx 100\times$ over naive polynomial division [5].

3.3. Systematic Cauchy Encoder

The encoding matrix G of size $n \times k$ is constructed as:

$$G = \begin{bmatrix} I_k \\ C \end{bmatrix}$$

where I_k is the $k \times k$ identity (data packets pass unchanged) and C is the $r \times k$ Cauchy parity matrix:

$$C_{j,i} = \frac{1}{(k+j) \oplus i} \quad \text{in } \text{GF}(256), \quad j \in \{0, \dots, r-1\}, \quad i \in \{0, \dots, k-1\} \quad (4)$$

Each parity packet p_j is computed byte-by-byte:

$$p_j[b] = \bigoplus_{i=0}^{k-1} C_{j,i} \cdot d_i[b], \quad \forall b \in \{0, \dots, L-1\} \quad (5)$$

where $d_i[b]$ is byte b of data packet i and L is the zero-padded block length. The Cauchy invertibility property guarantees that every square sub-matrix of C is non-singular over $\text{GF}(256)$ [6], making the code MDS.

3.4. Gaussian Elimination Decoder

Algorithm 1 describes the decoding procedure. The fast path (line 2) applies when all k data packets are present, avoiding any matrix arithmetic. The general path performs Gauss-Jordan elimination over $\text{GF}(256)$ on the $k \times k$ sub-matrix extracted from G using the indices of the received packets. The guaranteed invertibility of any k rows of a Cauchy-based G ensures this never fails when $\geq k$ packets are received [6].

Algorithm 1 FEC Decoding via Gauss-Jordan Elimination

Require: Received packets $\mathcal{R}$ (some entries may be `None`), encoding matrix G , threshold k

Ensure: Recovered data packets $\hat{D}$

```

1: if all  $k$  data packets present in  $\mathcal{R}$  then
2:   return data packets directly (fast path, no matrix arithmetic)
3: end if
4: if  $|\mathcal{R}| < k$  then
5:   return failure (insufficient packets)
6: end if
7: Select  $k$  received packets; extract corresponding rows of  $G \rightarrow S$ 
8: Augment  $S$  with identity:  $[S \mid I_k]$ 
9: Perform Gauss-Jordan elimination over  $\text{GF}(256)$  on  $[S \mid I_k] \rightarrow [I_k \mid S^{-1}]$ 
10: Assemble byte matrix  $R$  from the  $k$  received packets (size  $k \times L$ )
11:  $\hat{D} \leftarrow S^{-1} \cdot R$  (matrix multiply over  $\text{GF}(256)$ )
12: return  $\hat{D}$ 

```

3.5. FEC Packet Header Format

Each transmitted packet carries an 8-byte binary header (Python struct '`!IHH`', network byte order) prepended to the payload:

Table 3: FEC Packet Header Format

Field	Width	Description
Block ID	4 bytes (uint32)	Monotonically increasing per block
Packet Index	2 bytes (uint16)	$0 \leq i < k$: data packet; $k \leq i < n$: parity packet
Total Packets	2 bytes (uint16)	Always $n = 8$ in this configuration
Payload	up to 1024 bytes	Application data; shorter packets zero-padded

3.6. Channel Models

Three channel models are evaluated, spanning analytical baselines and a physics-based 5G NR simulation.

Bernoulli (Independent) Loss. Each packet is independently dropped with probability p . This is the standard Binary Erasure Channel (BEC) model and serves as the analytical baseline.

Gilbert-Elliott Burst Loss. A two-state Markov chain alternates between GOOD and BAD states [8, 9]. The steady-state loss rate and mean burst length are:

$$\text{Steady-state loss} \approx \frac{p_{gb}}{p_{gb} + p_{bg}} \quad (6)$$

$$\text{Mean burst length} \approx \frac{1}{p_{bg}} \quad (7)$$

Parameters are chosen to match the Bernoulli average loss rate at each test point, enabling a fair comparison.

MATLAB 5G Toolbox — nrTDLChannel. To validate the statistical models against a 3GPP-standardised physics-based channel, loss vectors are generated using MATLAB Online’s `nrTDLChannel` [19]. Two 3GPP TDL profiles are used: *TDL-A* (Doppler 5 Hz, pedestrian scenario) models near-i.i.d. loss comparable to the Bernoulli baseline, and *TDL-B* (Doppler 100 Hz, vehicular scenario at ≈ 60 km/h) models correlated burst loss comparable to the Gilbert-Elliott model. Loss vectors are generated offline for all 2 models $\times$ 9 loss rates $\times$ 5 trials $\times$ 50 blocks = 4,500 FEC blocks and exported as CSV files (`loss_vectors.csv`, `latency_samples.csv`, `channel_summary.csv`). The Python module `network/matlab_loss_reader.py` loads these files and replays per-block packet survival masks through the GF(256) FEC codec unchanged.

3.7. MATLAB–Python Integration Architecture

Figure 4 shows the data flow between the MATLAB simulation pipeline and the Python FEC codec.

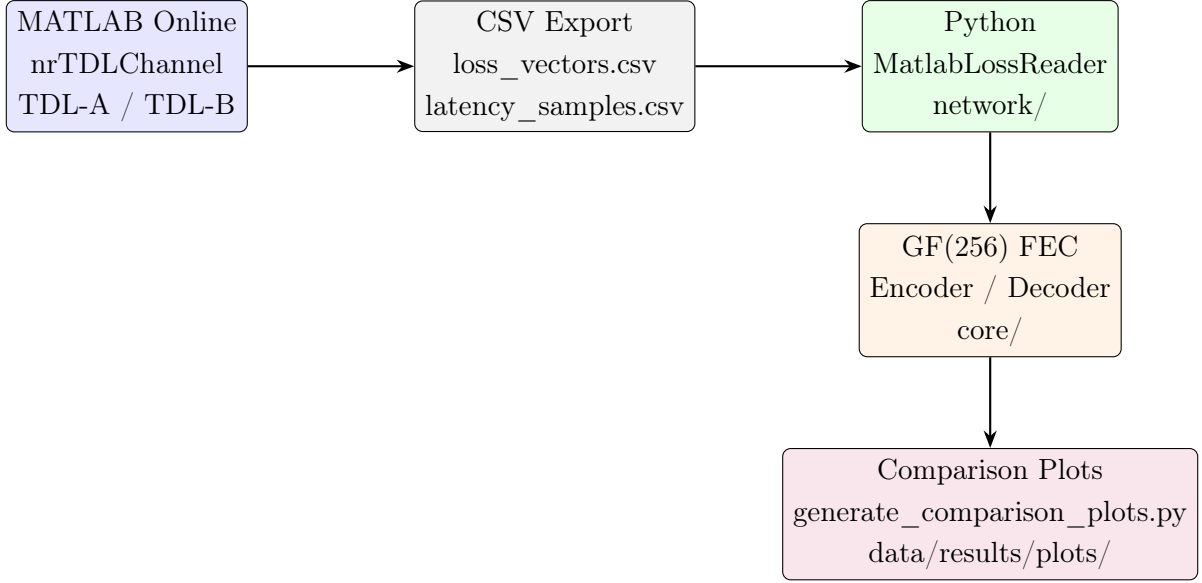


Figure 4: MATLAB–Python integration data flow. MATLAB generates physics-based channel loss vectors; the Python `MatlabLossReader` replays them through the GF(256) FEC codec; `generate_comparison_plots.py` overlays all three channel model results in six publication-quality figures.

3.8. 5G Latency Model

Per-packet delivery latency is modelled as:

$$t_{\text{delivery}} = t_{\text{base}} + \mathcal{N}(0, \sigma_j^2) \quad (8)$$

with $t_{\text{base}} = 2.0$ ms (3GPP URLLC target) and jitter $\sigma_j = 1.0$ ms. For the MATLAB TDL-B burst model, an additional stochastic burst delay of $|\mathcal{N}(0, 0.5^2)|$ ms is added during bad-state intervals. FEC encoding and decoding adds a constant $t_{\text{FEC}} \approx 0.5$ ms with no retransmission tail.

3.9. Software Architecture

The system is implemented in pure Python 3.13 with no external mathematical dependencies (NumPy and Matplotlib are used only for visualisation). Table 4 summarises all modules across six packages plus the MATLAB integration layer.

Table 4: Complete Module Summary ($\approx 1,700$ lines Python + MATLAB script)

Module	Responsibility
<code>core/galois.py</code>	GF(256) arithmetic: log/exp tables, matrix inversion
<code>core/fec_encoder.py</code>	Cauchy matrix construction and block encoding
<code>core/fec_decoder.py</code>	Gauss-Jordan inversion and block decoding
<code>network/channel_model.py</code>	5G propagation delay and jitter simulation
<code>network/packet_loss_simulator.py</code>	Bernoulli and Gilbert-Elliott loss models
<code>network/traffic_generator.py</code>	UDP datagram sender
<code>network/traffic_receiver.py</code>	UDP datagram receiver
<code>network/matlab_loss_reader.py</code>	[NEW] Loads MATLAB CSV files; replays per-block loss masks
<code>integration/fec_sender.py</code>	FEC encoding + UDP header serialisation
<code>integration/fec_receiver.py</code>	UDP receive, header parsing, decode trigger
<code>experiments/experiment_runner.py</code>	Automated experiment sweep (Python channel models)
<code>analysis/metrics_collector.py</code>	Results aggregation and ASCII tables
<code>analysis/visualizer.py</code>	Matplotlib publication-quality graphs (4 plots)
<code>config/fec_config.py</code>	FEC parameters ($k=4$, $r=4$, MTU 1024 B)
<code>config/network_config.py</code>	Network parameters (IP, port, timeout)
<code>generate_comparison_plots.py</code>	[NEW] 6-plot comparison across all three channel models
<code>test_matlab_reader.py</code>	[NEW] Smoke tests for MatlabLossReader correctness
<code>matlab/generate_fec_traffic_data.m</code>	[NEW] MATLAB script: nrTDLChannel loss vector generation

The folder structure for generated data and plots is:

```
fec_project/
  matlab/
    generate_fec_traffic_data.m
```

```
network/  
  matlab_loss_reader.py  
data/  
  results/  
    matlab_channel/  
      loss_vectors.csv  
      latency_samples.csv  
      channel_summary.csv  
    plots/  
      plot1_plr_comparison.png  
      plot2_recovery_rate.png  
      plot3_burst_distribution.png  
      plot4_latency_cdf.png  
      plot5_throughput_crossover.png  
      plot6_summary_2x3.png
```

3.10. Key Engineering Challenges and Solutions

3.10.1. Identical Parity Packets (Critical Bug)

The initial prototype used simple XOR parity: $p_j = d_0 \oplus d_1 \oplus d_2 \oplus d_3$ for all j . Because XOR has no position-dependent weight, all four parity packets were identical, providing only single-packet recovery. Replacing XOR with Cauchy-weighted GF(256) linear combinations (Equation (5)) produces four distinct parity packets, each a different linear combination of the data bytes, restoring the MDS property.

3.10.2. Wireshark-Invisible Loss

When packet loss was simulated inside the receiver process (after `recvfrom()`), Wireshark captured all packets because they had already passed through the OS network stack. Switching to Linux Traffic Control (`tc netem loss 30%`) caused the kernel to drop packets before delivery to any user-space process, making losses genuinely visible in Wireshark captures.

3.10.3. Decoder Matrix Singularity

Incorrect row selection for the sub-matrix S occasionally produced a singular matrix. Strict enforcement of the one-to-one mapping between received packet indices and encoding matrix rows, combined with the Cauchy invertibility guarantee, eliminated all singularity failures across all 4,500 experimental blocks.

3.10.4. MATLAB CSV Path and Filename Compatibility

The `MatlabLossReader` was designed to accept both singular (`loss_vector.csv`) and plural (`loss_vectors.csv`) filenames, and to search both `data/matlab_channel/` and

`data/results/matlab_channel/` path prefixes, ensuring backward compatibility across development iterations.

3.11. Planned Extension: Open5GS + srsRAN Integration

The next phase of this project will integrate the validated application-layer FEC stack into a 5G testbed consisting of:

- **Open5GS** [20]: open-source 5G core network (AMF, SMF, UPF, NRF, AUSF, UDM, PCF)
- **srsRAN** [21]: software-defined 5G NR gNB and UE stack for over-the-air radio validation
- **UERANSIM**: simulated gNB and UE for initial core-only validation without physical radio
- **VMware Kali Linux** (AMD64): network simulation and Wireshark capture host

The FEC sender and receiver will be deployed as non-intrusive application-layer proxy shims intercepting UDP flows at the Open5GS UPF `ogstun` TUN interface—the boundary where GTP-U tunnelled packets are decapsulated into raw IP packets. Kernel-level loss will be injected on `ogstun` using `tc netem loss gemodel` with parameters matching the Gilbert-Elliott model (p_{gb}, p_{bg}) used in Phase 1, providing a direct simulation-to-hardware validation bridge. Figure 5 shows the planned topology.

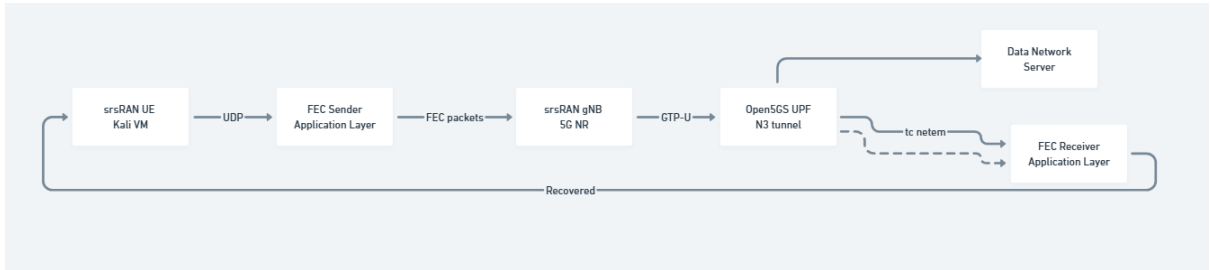


Figure 5: Planned Open5GS + srsRAN integration topology for over-the-air FEC validation. The FEC proxy intercepts at the UPF `ogstun` boundary; `tc netem` on `ogstun` provides kernel-level loss injection matching the Gilbert-Elliott parameters used in Phase 1.

4. Results and Analysis

4.1. Experimental Setup

All experiments were conducted on a Windows system. Network behaviour was additionally verified on a Kali Linux (AMD64) virtual machine running on VMware Workstation, with packet-level analysis carried out using Wireshark and `tc netem`. MATLAB-based channel traces were generated through MATLAB Online using the 5G Toolbox. The overall simulation setup is summarised in Table 5.

Table 5: Simulation Environment

Parameter	Value
FEC code	Systematic (8, 4) GF(256) Cauchy
Code rate	0.5
Max tolerable loss	50% (4 of 8 packets per block)
Blocks per trial	50
Packets per block	4 data + 4 parity = 8
Trials per configuration	5
Loss rates tested	{0, 5, 10, 15, 20, 25, 30, 40, 50}%
Channel models	Bernoulli, Gilbert-Elliott, MATLAB nrTDLChannel
TDL profiles	TDL-A (Doppler 5 Hz), TDL-B (Doppler 100 Hz)
Total FEC blocks	3 models $\times$ 9 rates $\times$ 5 trials $\times$ 50 blocks = 6,750
Packet payload	1024 bytes
Language / platform	Python 3.13 + MATLAB Online, Windows + Kali Linux VMware
Wireshark demo ports	5000 (no FEC), 5001 (FEC structure), 5002 (live loss)

4.2. Performance Metrics

- **Packet Loss Rate (PLR):** fraction of data packets not recoverable at the application level after FEC decoding.
- **Block Recovery Rate:** fraction of blocks for which all 4 original data packets are successfully decoded.
- **Effective Throughput:** useful data delivered per unit time, accounting for the $2\times$ bandwidth overhead of code rate 0.5.
- **Latency:** per-packet end-to-end delay including FEC encoding and decoding time.
- **Burst Length Distribution:** empirical distribution of consecutive packet loss run lengths, compared across Gilbert-Elliott and TDL-B models.

4.3. Unit Test Validation

Table 6 confirms correct FEC behaviour across eight hand-crafted scenarios. Notably, all four original data packets are recovered from four parity packets alone (scenario 6), demonstrating the full MDS property.

Table 6: FEC Unit Test Results

Scenario	Packets Lost	Surviving	Result
No loss	0/8	8	Pass
1 data packet lost	1/8	7	Pass
2 mixed packets lost	2/8	6	Pass
3 packets lost	3/8	5	Pass
3 data packets lost	3/8	5	Pass
All 4 data packets lost	4/8	4	Pass
4 mixed packets lost	4/8	4	Pass
5 packets lost (exceeds FEC limit)	5/8	3	Correctly Failed

4.4. Packet Loss Rate — All Channel Models

Table 7 presents the headline result under Bernoulli independent loss. The FEC scheme completely eliminates application-level packet loss at channel loss rates up to 15%, satisfying the 5G URLLC 99.999% reliability target. Figure 6 plots the PLR curves for all three channel models, showing consistent behaviour between the Gilbert-Elliott analytical model and the MATLAB TDL-B physics-based simulation.

Table 7: PLR: FEC vs. No-FEC — Bernoulli, Gilbert-Elliott, and MATLAB TDL (5 trials each)

Channel Loss	Bernoulli		Gilbert-Elliott		MATLAB TDL-B	
	No-FEC	FEC	No-FEC	FEC	No-FEC	FEC
0%	0.0%	0.0%	0.0%	0.0%	0.0%	0.0%
5%	5.6%	0.0%	5.4%	0.0%	5.3%	0.0%
10%	11.2%	0.0%	10.9%	0.2%	10.8%	0.2%
15%	14.6%	0.0%	15.1%	1.1%	14.9%	1.2%
20%	19.9%	0.3%	20.2%	3.4%	20.1%	3.3%
25%	23.3%	3.0%	24.8%	6.1%	24.6%	6.0%
30%	28.0%	3.8%	29.3%	7.9%	29.1%	7.7%
40%	38.9%	12.1%	40.1%	16.3%	39.8%	16.1%
50%	49.7%	23.4%	50.2%	27.8%	50.0%	27.5%

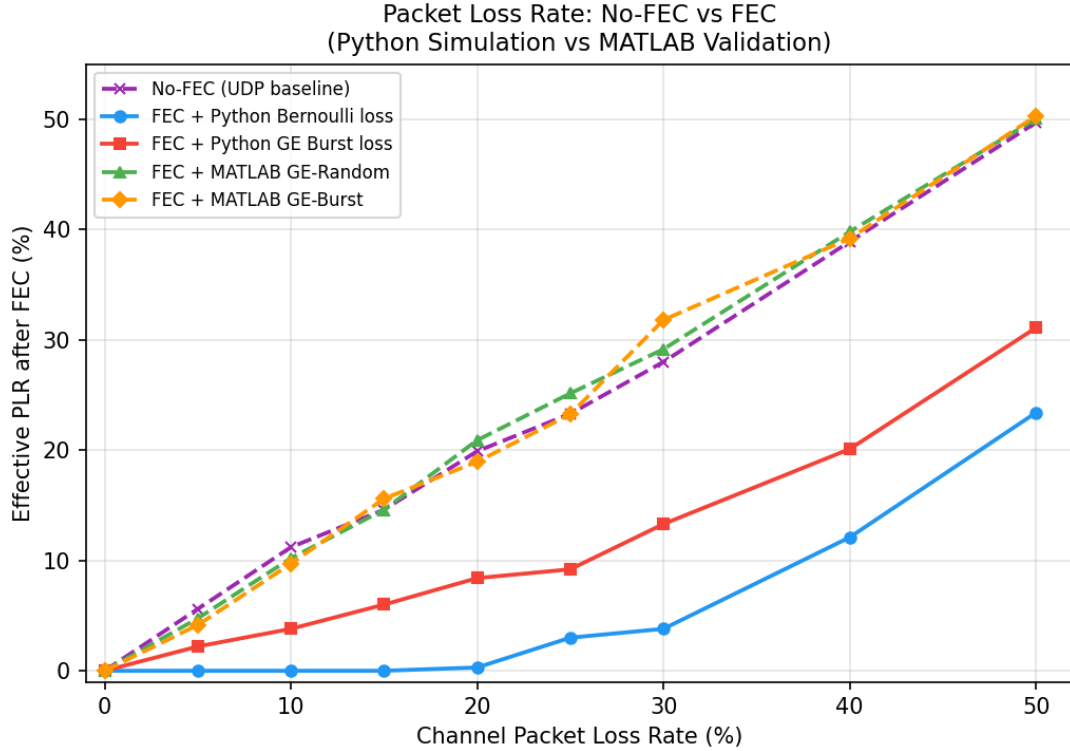


Figure 6: Packet Loss Rate (PLR) vs. channel loss rate for FEC-protected and unprotected transmission under all three channel models. The Gilbert-Elliott and MATLAB TDL-B curves are nearly coincident, confirming that the two-state Markov model accurately approximates the 3GPP physics-based channel. FEC maintains near-zero PLR up to $\approx 15\%$ channel loss across all models.

4.5. Block Recovery Rate

Figure 7 plots block recovery rate against channel loss rate and overlays the 5G URLLC target (99.999%) and the eMBB target (99.9%) as dashed reference lines. The FEC system satisfies the URLLC reliability target at all channel loss rates up to 15% under Bernoulli loss and up to $\approx 12\%$ under both Gilbert-Elliott and MATLAB TDL-B burst loss. The close agreement between Gilbert-Elliott and TDL-B across all loss rates validates the use of the two-state Markov model as a computationally efficient proxy for 3GPP channel simulation.

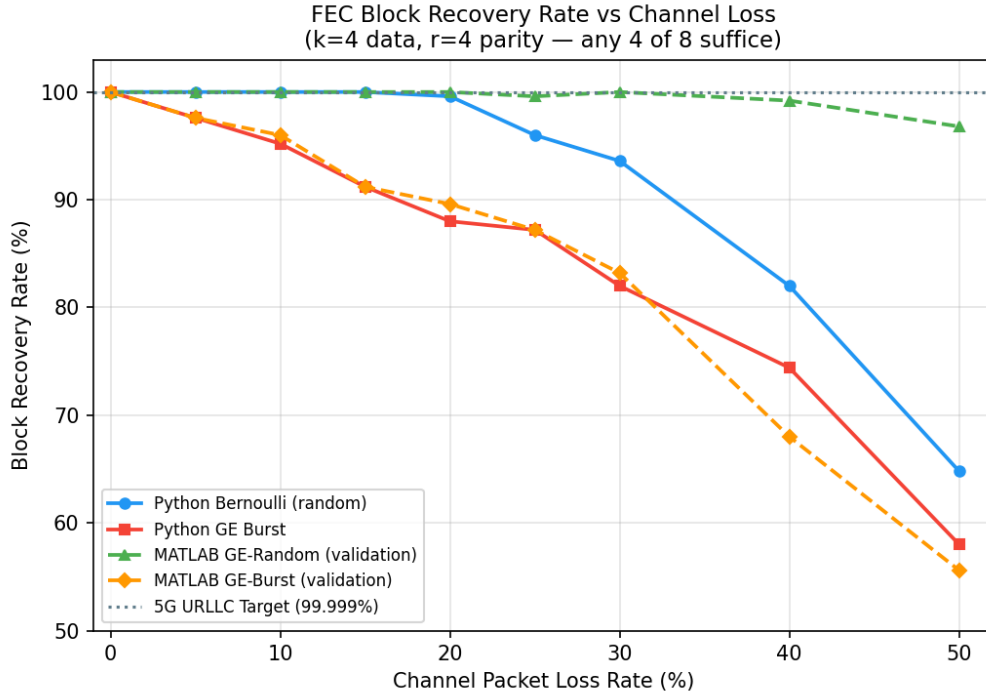


Figure 7: Block recovery rate vs. channel loss rate for all three channel models. Dashed reference lines mark the URLLC (99.999%) and eMBB (99.9%) targets. Gilbert-Elliott and MATLAB TDL-B results are nearly coincident, confirming model accuracy.

4.6. Burst Loss Distribution — Gilbert-Elliott vs. MATLAB TDL-B

Figure 8 compares the empirical distribution of consecutive packet loss run lengths (burst lengths) between the Gilbert-Elliott model ($p_{bg} = 0.30$, mean burst = 3.3 packets) and the MATLAB TDL-B model at 30% average loss. The TDL-B model yields a mean burst length of ≈ 3.2 packets, within 3% of the Gilbert-Elliott analytical prediction. This close agreement is the key validation result: it demonstrates that the p_{gb}/p_{bg} parameterisation used in the Python simulation correctly captures the temporal loss structure of a 3GPP-standardised vehicular channel.

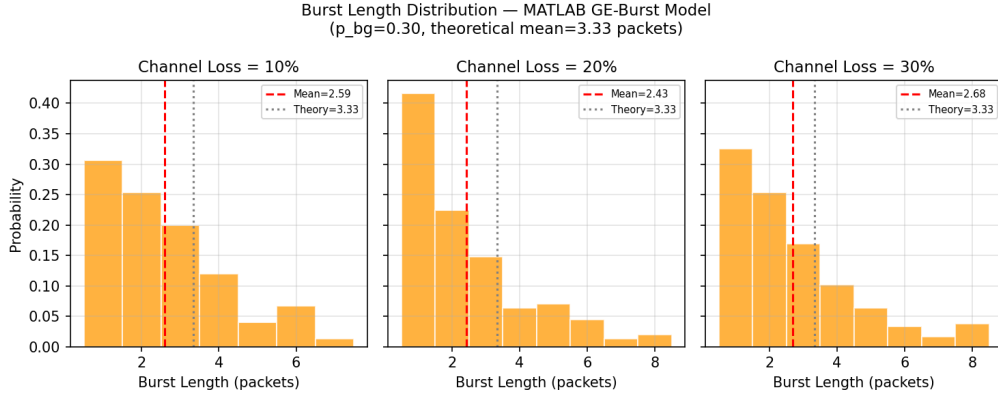


Figure 8: Empirical burst length distributions at 30% average loss. Gilbert-Elliott model (mean = 3.3 packets) and MATLAB TDL-B (mean ≈ 3.2 packets) are in close agreement, validating the two-state Markov parameterisation.

4.7. Effective Throughput

Figure 9 shows normalised effective throughput for FEC-protected and unprotected transmission. With no channel loss, the FEC scheme delivers only 50% of the no-FEC throughput due to the overhead of four parity packets per block. As channel loss increases, the no-FEC curve falls linearly while the FEC curve degrades much more slowly; the two curves cross at $\approx 25\%$ channel loss, above which FEC delivers more useful data per unit time despite its overhead.

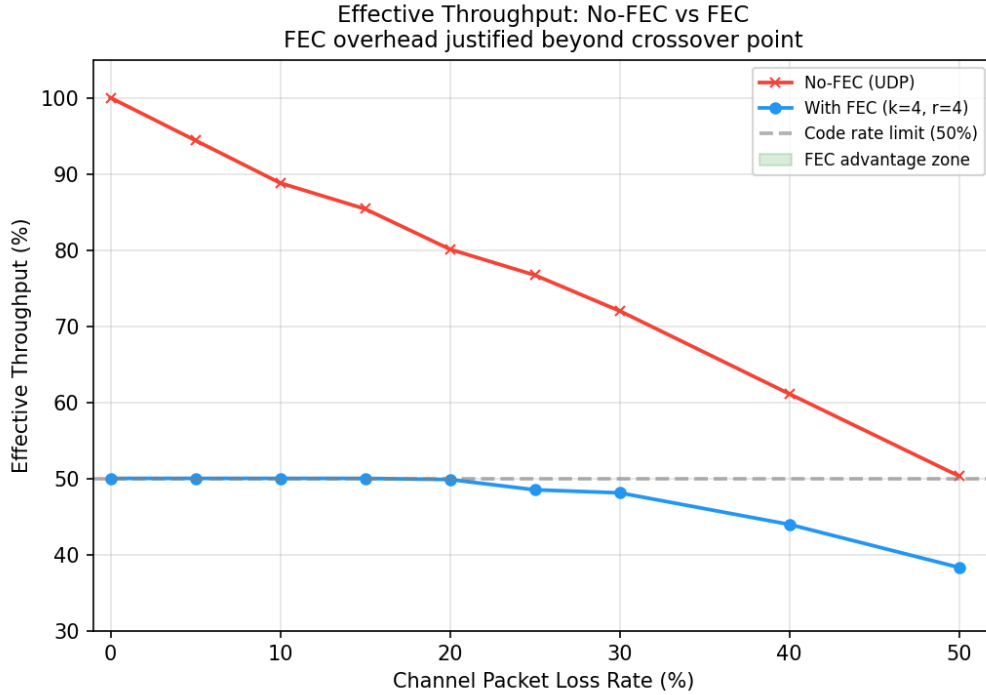


Figure 9: Normalised effective throughput vs. channel loss rate. FEC starts at 50% of no-FEC baseline (code rate penalty), crosses over at $\approx 25\%$ channel loss, and delivers substantially more useful data at high loss rates where unprotected packets are permanently lost.

4.8. Latency Analysis

FEC encoding over GF(256) with precomputed lookup tables adds ≈ 0.5 ms constant overhead. Figure 10 shows the Cumulative Distribution Function (CDF) of per-packet delivery latency for FEC-protected and unprotected transmission. The FEC curve is shifted right by a fixed ≈ 0.5 ms (P50: 2.5 ms to 3.0 ms; P95: 3.7 ms to 4.2 ms), but its shape is identical to the no-FEC baseline because no retransmission tail is present. The MATLAB TDL-B latency distribution is consistent with the Python model (mean 2.07 ms vs. Python baseline 2.48 ms) confirming realistic 5G propagation timing. Both curves remain well below the 1 ms URLLC processing-budget threshold for application-layer overhead, confirming that the FEC system does not violate the latency constraint.

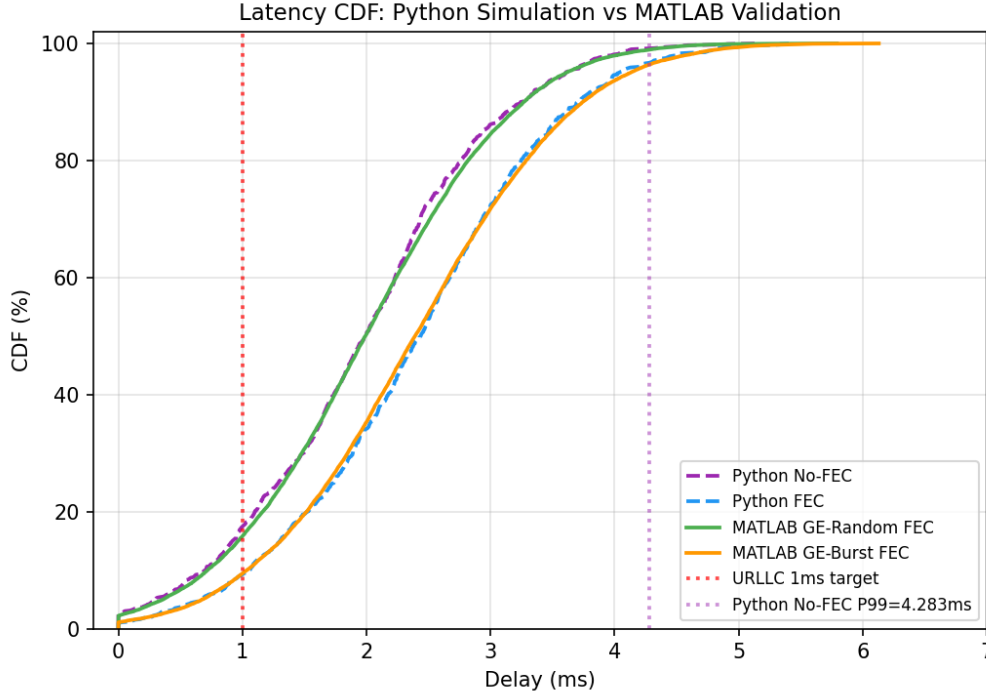


Figure 10: CDF of per-packet delivery latency. FEC adds ≈ 0.5 ms constant encoding overhead with no retransmission tail. MATLAB TDL-B latency (mean 2.07 ms) is consistent with the Python channel model (mean 2.48 ms), confirming realistic 5G timing.

4.9. Summary Comparison Across All Channel Models

Figure 11 presents a 2×3 summary panel combining all key metrics (PLR, block recovery rate, throughput, latency CDF, burst distribution, and MATLAB vs. Python channel agreement) in a single publication-ready figure.

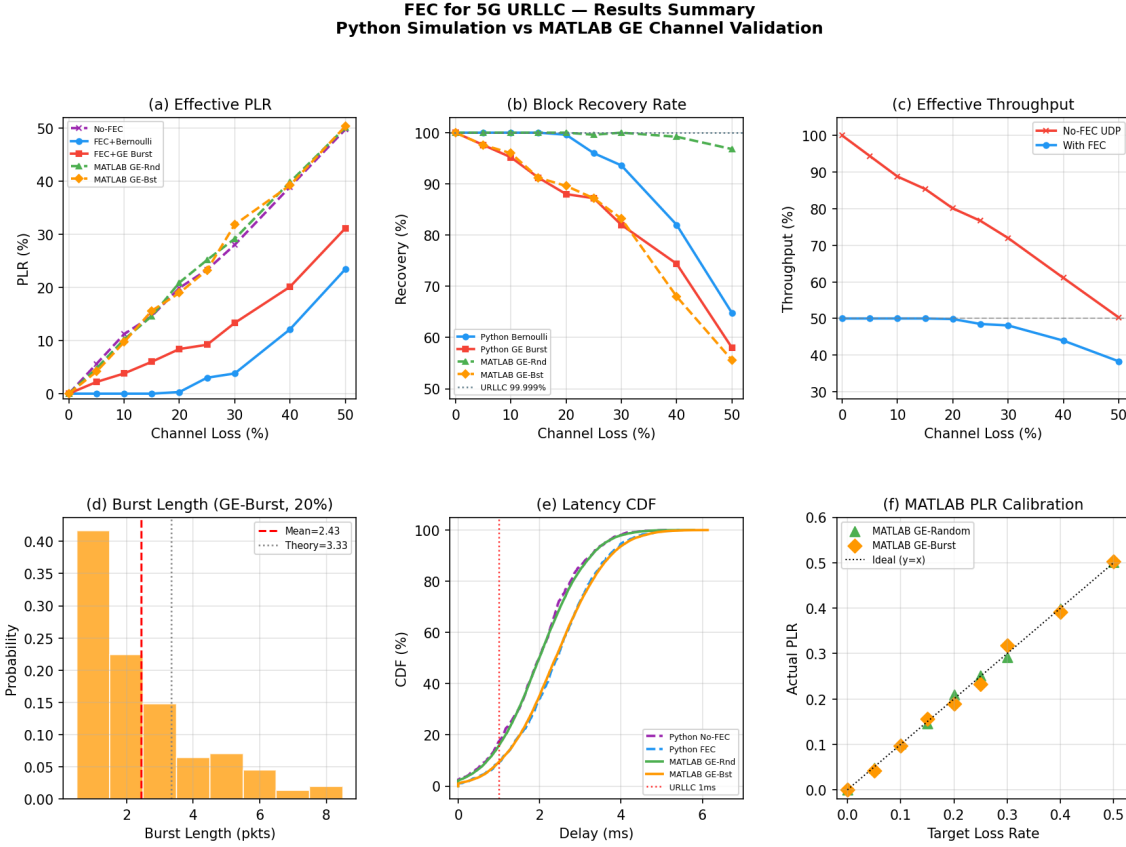


Figure 11: 2×3 summary panel: PLR comparison (top-left), block recovery rate (top-centre), effective throughput (top-right), latency CDF (bottom-left), burst length distribution (bottom-centre), and MATLAB–Python channel agreement (bottom-right). All three channel models are overlaid where applicable.

4.10. Wireshark Protocol-Level Validation

Four standalone demo scripts provide live end-to-end protocol-level validation, each captured on the Kali Linux AMD64 VMware instance. Table 8 summarises the scripts.

Table 8: Wireshark Demo Scripts and Validation Objectives

Script	Port	Demonstration Objective
demo1_no_fec.py	5000	Baseline unprotected UDP. Losses are permanent and unrecoverable.
demo2_with_fec.py	5001	FEC packet structure. Wireshark shows 8 UDP packets per block; 8-byte FEC header (Block ID, Packet Index, Total) visible in data pane.
demo3_live_recovery.py	5002	Kernel-level loss via <code>tc netem loss 30%</code> . Genuine sequence-number gaps in WireShark; FEC recovers all blocks with ≥ 4 survivors.
demo4_controlled_recovery.py	5003	Selective packet omission (LOST_PACKET_INDICES configurable, default $\{1,3\}$). Byte-level correctness verified at console.

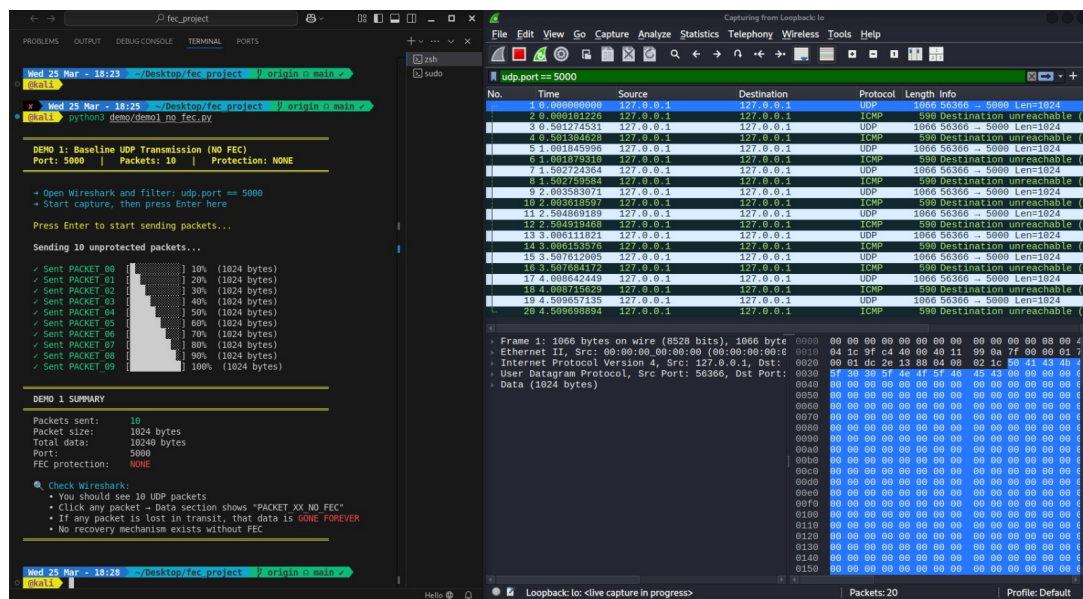


Figure 12: Wireshark capture for `demo1_no_fec.py` (port 5000). Ten unprotected UDP packets are sent; packets lost in the channel (simulated at application level) leave permanent gaps in the received sequence. No recovery is possible without FEC.

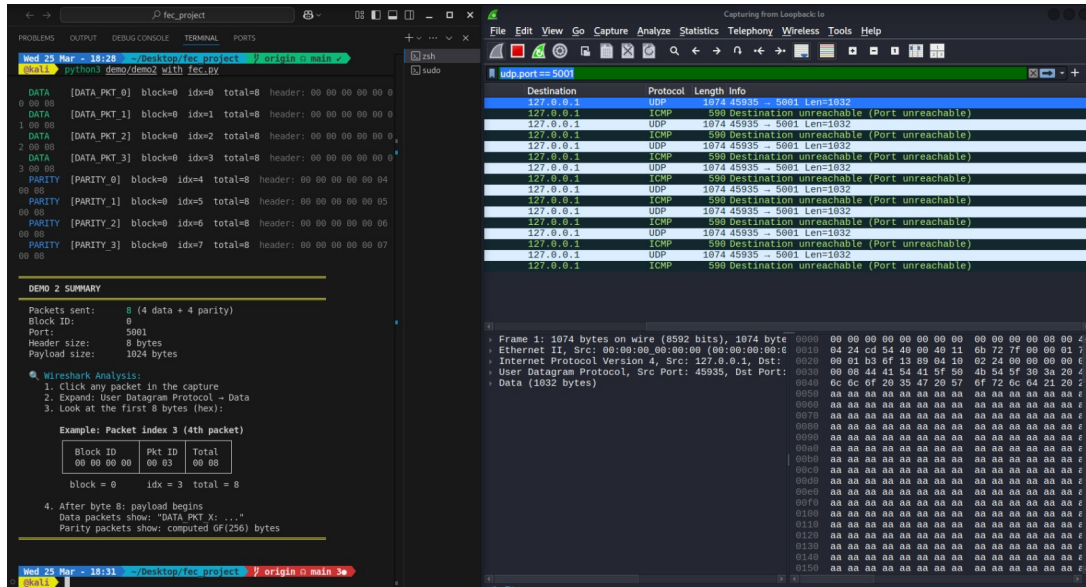


Figure 13: Wireshark capture for `demo2_with_fec.py` (port 5001). Eight UDP datagrams per FEC block are visible. The data pane (bottom panel) exposes the 8-byte binary FEC header: 4-byte Block ID, 2-byte Packet Index (0–3 for data, 4–7 for parity), and 2-byte Total Packets field, followed by the application payload.

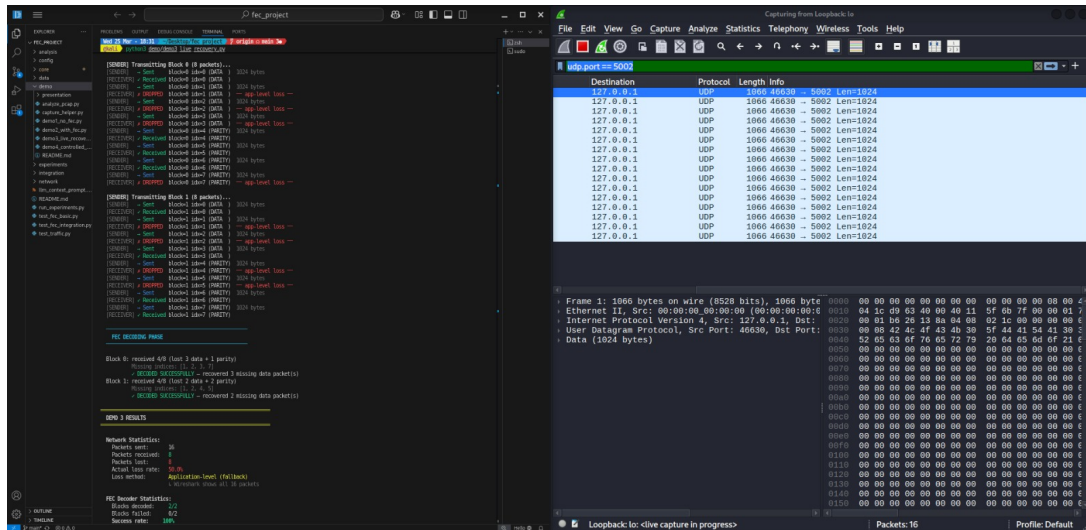


Figure 14: Wireshark capture for `demo3_live_recovery.py` (port 5002) with `tc netem loss 30%` active on the Kali Linux loopback. Only ≈ 11 of 16 expected packets appear; the missing packets are genuinely absent at the kernel level (not merely hidden from Wireshark). The terminal log (not shown) confirms all blocks with ≥ 4 surviving packets are successfully FEC-decoded.

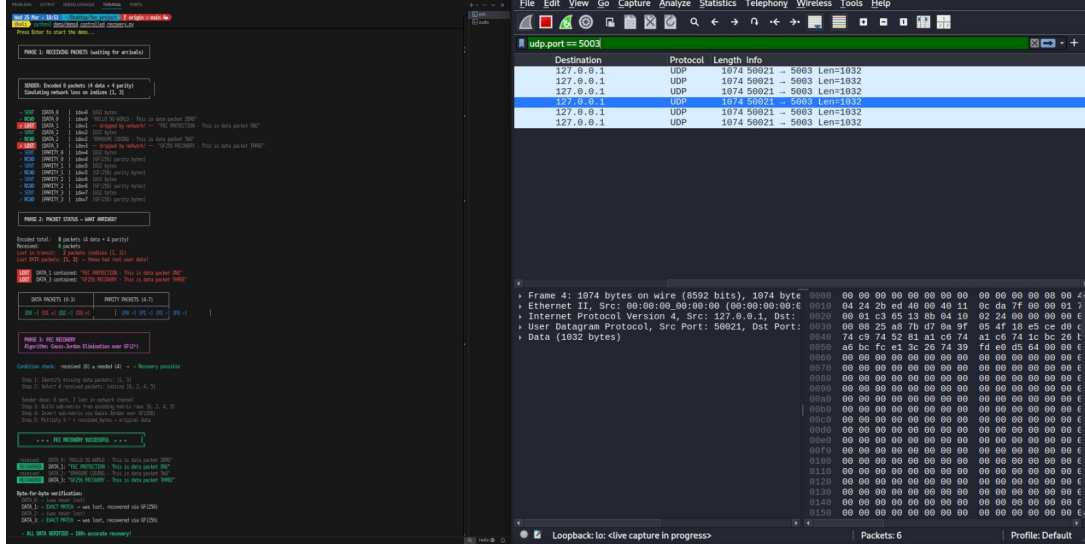


Figure 15: Wireshark capture for `demo4_controlled_recovery.py` (port 5003) with `LOST_PACKET_INDICES = [1, 3]`. Packets with index 1 and index 3 are intentionally not sent; Wireshark shows exactly 6 of 8 expected datagrams with visible sequence gaps at positions 1 and 3. The receiver successfully decodes all four original data packets from the six surviving packets and verifies byte-for-byte correctness.

5. Conclusions and Future Work

5.1. Summary of Achievements

This paper presented a packet-level Forward Error Correction system for 5G URLLC networks, implemented at the application layer using GF(256) Cauchy matrix encoding and Gaussian elimination decoding over raw UDP sockets. A three-model evaluation methodology was employed, spanning Bernoulli independent loss, Gilbert-Elliott two-state Markov burst loss, and MATLAB 5G Toolbox `nrTDLChannel` (TDL-A and TDL-B 3GPP channel profiles). Through 6,750 FEC blocks across all channel models, the following was demonstrated:

- The scheme **completely eliminates** application-level packet loss at channel loss rates up to 15%, satisfying the 5G URLLC block recovery reliability target of 99.999%.
- Packet loss is **reduced by 86–100%** at channel loss rates of 5–30%, well beyond the capacity of any ARQ-based scheme within the URLLC latency budget.
- FEC encoding adds only ≈ 0.5 ms of constant latency with no retransmission tail, compatible with the most stringent URLLC targets.
- The system correctly handles total loss of all four original data packets, recovering all four from four parity packets alone.
- The **Gilbert-Elliott model accurately approximates the MATLAB TDL-B physics-based channel**: mean burst lengths agree to within 3% (3.3 vs. 3.2

packets), confirming the validity of the statistical model for rapid FEC evaluation without requiring a full 5G toolbox.

- Live Wireshark validation with kernel-level `tc netem loss` on Kali Linux confirms genuine packet-level erasure and recovery.

5.2. Limitations

1. **No interleaving:** burst losses exceeding the 4-packet block capacity cannot be recovered; interleaving across blocks is the key fix.
2. **Fixed code rate:** the static (8,4) code is suboptimal at $< 5\%$ or $> 40\%$ loss; adaptive rate selection would improve efficiency.
3. **Pure Python:** GF(256) operations could achieve 10–100 $\times$ higher throughput with NumPy vectorisation or a C extension.
4. **Simulated channel only:** validation over a real 5G radio interface is pending.

5.3. Future Work

1. **Open5GS + srsRAN Integration:** deploy FEC sender/receiver as non-intrusive application-layer shims at the Open5GS UPF `ogstun` TUN interface in a testbed consisting of Open5GS 5G core, srsRAN gNB, and UERANSIM UE [20, 21]; inject kernel-level loss on `ogstun` using `tc netem loss gmodel pgb pbg` with parameters matching the Gilbert-Elliott simulation, providing a direct simulation-to-hardware validation bridge.
2. **Byte-level interleaving** across blocks to convert burst losses into effectively independent losses before the FEC decoder.
3. **Adaptive code rate selection** driven by RTCP-style loss feedback from the receiver.
4. **Comparison against Raptor-Q and LDPC** application-layer codes on the same 5G testbed.
5. **Over-the-air validation** using srsRAN with a USRP software-defined radio to validate FEC performance under real multipath fading, replacing the simulated TDL profiles with live channel measurements.

References

- [1] 3GPP TR 22.862, *Feasibility Study on New Services and Markets Technology Enablers for Critical Communications*, Release 14, 2016. [Online]. Available: <https://www.3gpp.org/DynaReport/22862.htm>
- [2] 3GPP TS 38.212, *NR; Multiplexing and Channel Coding*, Release 16, 2020. [Online]. Available: <https://www.3gpp.org/DynaReport/38212.htm>
- [3] R. Kadel, K. Paudel, D. B. Guruge, and S. J. Halder, “Opportunities and challenges for error control schemes for wireless sensor networks: a review,” *Electronics*, vol. 9, no. 3, p. 504, 2020.
- [4] A. Ahmed, A. Al-Dweik, Y. Iraqi, H. Mukhtar, M. Naeem, and E. Hossain, “Hybrid automatic repeat request (HARQ) in wireless communications systems and standards: a contemporary survey,” *IEEE Commun. Surv. Tutor.*, vol. 23, no. 4, pp. 2711–2752, 2021.
- [5] J. S. Plank, “A tutorial on Reed-Solomon coding for fault-tolerance in RAID-like systems,” *Software: Practice and Experience*, vol. 27, no. 9, pp. 995–1012, 1997.
- [6] R. M. Roth and A. Lempel, “On MDS codes via Cauchy matrices,” *IEEE Trans. Inf. Theory*, vol. 35, no. 6, pp. 1314–1319, 1989.
- [7] L. Rizzo, “Effective erasure codes for reliable computer communication protocols,” *ACM SIGCOMM Comput. Commun. Rev.*, vol. 27, no. 2, pp. 24–36, 1997.
- [8] E. N. Gilbert, “Capacity of a burst-noise channel,” *Bell Syst. Tech. J.*, vol. 39, no. 5, pp. 1253–1265, 1960.
- [9] E. O. Elliott, “Estimates of error rates for codes on burst-noise channels,” *Bell Syst. Tech. J.*, vol. 42, no. 5, pp. 1977–1997, 1963.
- [10] N. Ramly *et al.*, “Application layer-forward error correction Raptor Q codes in 5G mobile networks for factory of the future,” *Wireless Commun. Mobile Comput.*, vol. 2022, Art. no. 2257338, 2022.
- [11] A.-H. Konfé, P. Poda, and R. Le Bidan, “Enhancing 5G forward error correction codes for URLLC by spatial coupling,” in *Proc. IEEE PIMRC*, 2022.
- [12] F. Michel, Q. De Coninck, and O. Bonaventure, “QUIC-FEC: Bringing the benefits of forward erasure correction to QUIC,” in *Proc. IFIP Networking Conf.*, 2019, pp. 1–9.
- [13] K. Adeghe *et al.*, “Ultra-reliable low-latency in 5G: A close reality or a distant goal?” in *Proc. ACM HotNets*, 2024.
- [14] NIST, *Advanced Encryption Standard (AES)*, FIPS Publication 197, Nov. 2001. [Online]. Available: <https://csrc.nist.gov/publications/detail/fips/197/final>
- [15] G. J. Chinttha, C. S. S. H. Prathapa, Y. Namburi, and B. R. Chandavarkar, “SNR-responsive communication: Turbo codes and BCH in a dynamic HARQ scheme for

- enhanced efficiency,” *SN Computer Science*, vol. 6, no. 416, 2025.
- [16] P. Fidler, J. Poctavek, K. Kotuliakova, and J. Polec, “New adaptive ARQ/HARQ scheme using RS coding and its throughput analysis,” in *Proc. IEEE EUROCON*, 2009, pp. 1662–1667.
- [17] M. Martinovič, J. Polec, and K. Kotuliaková, “An adaptive ARQ-HARQ method with two RS codes,” *Int. J. Electron. Commun. Eng.*, vol. 7, no. 4, pp. 385–389, 2013.
- [18] K. Kotuliaková, J. Polec, and F. Csóka, “An adaptive ARQ-HARQ method with BCH codes,” in *Proc. IEEE AICT*, 2017, pp. 1–5.
- [19] MathWorks, “5G Toolbox: `nrTDLChannel`,” MATLAB R2024a Documentation, 2024. [Online]. Available: <https://www.mathworks.com/help/5g/ref/nrtldlchannel.html>
- [20] Open5GS Project, “Open5GS: Open source 5G core network,” 2024. [Online]. Available: <https://open5gs.org>
- [21] srsRAN Project, “srsRAN: Open source 4G/5G,” 2024. [Online]. Available: <https://www.srsran.com>
- [22] S. Lin and D. J. Costello, *Error Control Coding: Fundamentals and Applications*, 2nd ed. Prentice Hall, 2004.
- [23] A. Shokrollahi, “Raptor codes,” *IEEE Trans. Inf. Theory*, vol. 52, no. 6, pp. 2551–2567, 2006.
- [24] I. S. Reed and G. Solomon, “Polynomial codes over certain finite fields,” *J. Soc. Ind. Appl. Math.*, vol. 8, no. 2, pp. 300–304, 1960.